

SQL – DQL

(Data Query Language)

List of Main DQL Operations

The main DQL operations are:

1. [SELECT](#) — Specifies which columns or expressions to retrieve.

[FROM](#) — Specifies the table or tables to retrieve data from.

2. [DISTINCT](#) — Removes duplicate rows from the result.

3. [WHERE](#), [LOGICAL OPERATORS](#), [BETWEEN](#) — Filters rows based on a condition.

4. [IN](#), [LIKE](#) and [Wildcards](#) — Checks whether a value matches any value in a list or subquery.

5. [Working With NULL](#) – Determine when a value is missing.

6. [ORDER BY](#) (ASC, DESC)— Sorts the query results.

7. [LIMIT](#) — Restricts the number of rows returned in MySQL.

8. [Aggregate Functions](#) - perform calculations using multiple rows.

9. [GROUP BY](#) — Groups rows that have the same values.

10. [HAVING](#) — Filters groups created by GROUP BY.

11. [Logical Ordering of SQL Query Executions](#)

12. [JOIN](#) (INNER JOIN)— Combines related rows from two or more tables.

[LEFT JOIN](#) — Returns all rows from the left table and matching rows from the right table.

[RIGHT JOIN](#) — Returns all rows from the right table and matching rows from the left table.

13. [Subqueries](#) — a query contained inside another query.

14. [Derived Tables](#) – a temporary result set.

15. [Common Table Expression \(CTE\)](#) - a temporary named result set used within a SQL query.

1. SELECT and FROM

Note: Use 'SQL DQL Scratchpad.sql' to create the database, tables, and insert values.

To retrieve all columns, use the asterisk *:

```
SELECT *  
FROM Student;
```

Instead of displaying every column in a table, we can specify only the columns we want.

Example:

```
SELECT ID, Name, Major  
FROM Student;
```

This retrieves only the StudentID, Name, and Major columns.

2. DISTINCT

The DISTINCT keyword removes duplicate rows from the query result.

Example:

```
SELECT Major  
FROM Student;
```

This might produce:

Computer Science
Biology
Business
Computer Science
Mathematics
Business
Computer Science
Biology
Mathematics
Computer Science
Business
Biology

Using DISTINCT:

```
SELECT DISTINCT Major  
FROM Student;
```

would produce:

Computer Science
Biology
Business
Mathematics

3. WHERE

The WHERE clause is used to select only rows that meet a particular condition.

Example:

```
SELECT *  
FROM Student  
WHERE Major = 'Computer Science';
```

Only students whose major is Computer Science will be displayed.
Common comparison operators include:

Another Example:

```
SELECT StudentID  
FROM Enrollment  
WHERE Grade <= 'B';
```

Operator	Meaning
=	Equal to
<>	Not equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Logical Operators

Multiple conditions can be combined using logical operators.

The most common are:

AND, OR, and NOT

AND

Both conditions must be true.

```
SELECT *  
FROM Student  
WHERE Major = 'Computer Science'  
AND GPA >= 3.0;
```

OR

At least one condition must be true.

```
SELECT *  
FROM Student  
WHERE Major = 'Computer Science'  
OR Major = 'Biology';
```

NOT

NOT reverses a condition.

```
SELECT *  
FROM Student  
WHERE NOT Major = 'Computer Science';
```

Parentheses can be used when combining several conditions.

```
SELECT *  
FROM Student  
WHERE GPA >= 3.0  
AND (Major = 'Computer Science'  
OR Major = 'Biology');
```

BETWEEN

BETWEEN checks whether a value falls within a range (inclusive).

Example:

```
SELECT *  
FROM Student  
WHERE GPA BETWEEN 3.0 AND 4.0;
```

This includes both 3.0 and 4.0.

The equivalent condition would be:

```
SELECT *  
FROM Student  
WHERE GPA >= 3.0  
AND GPA <= 4.0;
```

4. IN

The IN operator checks whether a value matches one of several possible values.

Instead of writing:

```
SELECT *  
FROM Student  
WHERE Major = 'Computer Science'  
OR Major = 'Biology'  
OR Major = 'Business';
```

we can write:

```
SELECT *  
FROM Student  
WHERE Major IN ('Computer Science',  
                'Biology',  
                'Business');
```

IN is often easier to read when several possible values are being checked.

LIKE and Wildcards

The LIKE operator is used for pattern matching.
Two commonly used wildcard characters are:

Wildcard	Meaning
%	Zero or more characters
_	Exactly one character

Example:

```
SELECT *  
FROM Student  
WHERE Name LIKE 'A%';
```

This retrieves students whose names begin with A.

Examples:

```
SELECT *  
FROM Student  
WHERE Name LIKE '%son';
```

Retrieves names ending in son.

```
SELECT *  
FROM Student  
WHERE Name LIKE '%le%';
```

Retrieves names containing le.

```
SELECT *  
FROM Student  
WHERE Name LIKE 'J_n%';
```

Retrieves names that starts with 'J' and the third letter is an 'n' .
Examples might include:
'Jun' and 'Jan'

5. Working with NULL

NULL means that a value is missing, unknown, or not available.

To check for **NULL**, use:
IS NULL

Example:

```
SELECT *  
FROM Student  
WHERE Major IS NULL;
```

To find values that are not NULL:

```
SELECT *  
FROM Student  
WHERE Major IS NOT NULL;
```

Do not write:

WHERE Major = NULL;

This will not work as expected because **NULL** represents an unknown value.

6. ORDER BY

The ORDER BY clause sorts the rows returned by a query.

Example:

```
SELECT *  
FROM Student  
ORDER BY Name;
```

By default, the order is ascending.

You may explicitly write:

```
ORDER BY Name ASC;
```

For descending order:

```
ORDER BY Name DESC;
```

You can also sort using multiple columns.

```
SELECT *  
FROM Student  
ORDER BY Major, Name;
```

The rows will first be sorted by Major. Students within the same major will then be sorted by Name.

7. LIMIT

In MySQL, LIMIT restricts the number of rows returned.

Example:

```
SELECT *  
FROM Student  
LIMIT 5;
```

Only the first five rows will be returned.

It is often combined with ORDER BY.

```
SELECT *  
FROM Student  
ORDER BY GPA DESC  
LIMIT 5;
```

This displays the five students with the highest GPAs.

8. Aggregate Functions

Aggregate functions perform calculations using multiple rows and return a summarized result.

Common aggregate functions include:

COUNT()

Counts rows.

```
SELECT COUNT(*)  
FROM Student;
```

SUM()

Adds numeric values.

```
SELECT SUM(Credits)  
FROM Course;
```

AVG()

Calculates an average.

```
SELECT AVG(GPA)  
FROM Student;
```

MIN()

Finds the smallest value.

```
SELECT MIN(GPA)  
FROM Student;
```

MAX()

Finds the largest value.

```
SELECT MAX(GPA)  
FROM Student;
```

9. GROUP BY

GROUP BY creates groups of rows that share the same value. Aggregate functions can then be applied to each group.

Example:

```
SELECT Major,  
    COUNT(*) AS NumberOfStudents  
FROM Student  
GROUP BY Major;
```

The result might look like:

Computer Science	15
Biology	10
Business	12

Instead of counting all students together, SQL counts the students within each major.

Another example:

```
SELECT Major,  
    ROUND(AVG(GPA), 2) AS 'Average GPA'  
FROM Student  
GROUP BY Major;
```

Note that this script is invalid:

```
SELECT *  
FROM Student  
GROUP BY Major;
```

It is invalid because GROUP BY Major creates one group per major, but SELECT * asks SQL to return all columns. For columns like ID, Name, and GPA, SQL would not know **which student's value to display** for each major.

10. HAVING

HAVING filters grouped results. It is commonly used together with GROUP BY.

Example:

```
SELECT Major,  
       COUNT(*) AS NumberOfStudents  
FROM Student  
GROUP BY Major  
HAVING COUNT(*) >= 3;
```

Only majors with at least 3 students will be displayed.

A useful distinction is:

WHERE → filters rows before grouping

HAVING → filters groups after GROUP BY

Example:

```
SELECT Major, AVG(GPA)  
FROM Student  
WHERE GPA IS NOT NULL  
GROUP BY Major  
HAVING AVG(GPA) >= 3.0;
```

11. A logical order of SQL Query Execution

1. FROM / JOIN
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. ORDER BY
7. LIMIT

For this query:

```
SELECT Major,  
        COUNT(*) AS NumberOfStudents  
FROM Student  
GROUP BY Major  
HAVING COUNT(*) >= 3;
```

The logical execution order is:

1. FROM Student
SQL first retrieves the rows from the Student table.
2. GROUP BY Major
The rows are grouped according to each Major.
3. HAVING COUNT(*) >= 3
SQL keeps only the groups that contain at least 3 students.
4. SELECT Major, COUNT(*) AS NumberOfStudents
Finally, SQL displays the major and the number of students in each qualifying group.

So although SELECT is written first, it is logically processed after FROM, GROUP BY, and HAVING.

SQL is generally easier to learn initially, but advanced SQL can become conceptually difficult because queries are not always processed in the order they are written.

12. JOIN

A JOIN combines rows from two or more related tables.

Suppose we have:

Student

StudentID

Name

Enrollment

StudentID

CourseID

We can combine the tables using:

```
SELECT Student.ID,  
       Student.Name,  
       Enrollment.CourseID  
FROM Student  
JOIN Enrollment  
  ON Student.ID = Enrollment.StudentID;
```

The ON clause describes how rows in the two tables are related.

JOIN without another keyword normally means INNER JOIN.

Therefore:

JOIN

and

INNER JOIN

mean the same thing.

An inner join returns rows only when a matching row exists in both tables.

LEFT JOIN

A LEFT JOIN returns:

all rows from the left table matching rows from the right table. If there is no matching row, the columns from the right table contain NULL.

Example:

```
SELECT Student.ID,  
       Student.Name,  
       Enrollment.CourseID  
FROM Student  
LEFT JOIN Enrollment  
ON Student.ID = Enrollment.StudentID;
```

All students will appear, even students who are not enrolled in a course. The table written before LEFT JOIN is the **left table**.

In this example:

Student
is the left table.

For example: Show all courses that have no students enrolled.

```
SELECT Course.ID,  
       Course.Name  
FROM Course  
LEFT JOIN Enrollment  
ON Course.ID = Enrollment.CourseID  
WHERE Enrollment.StudentID IS NULL;
```

RIGHT JOIN

A RIGHT JOIN does the opposite of a LEFT JOIN.

It returns:

all rows from the right table

matching rows from the left table

Example:

```
SELECT Student.ID,  
       Student.Name,  
       Enrollment.CourseID  
FROM Student  
RIGHT JOIN Enrollment  
ON Student.ID = Enrollment.StudentID;
```

The table written after RIGHT JOIN is the right table.

In many situations, a RIGHT JOIN can be rewritten as a LEFT JOIN simply by reversing the table order. Because of this, many developers primarily use LEFT JOIN.

Joining More Than Two Tables

Queries frequently combine several tables.

For example:

Student

Enrollment

Course

This query follows the relationships:

Student

|

Enrollment

|

Course

A student enrolls in a course through the Enrollment table. The query may be:

```
SELECT Student.Name AS StudentName,  
       Course.Name AS CourseName  
FROM Student  
JOIN Enrollment  
  ON Student.ID = Enrollment.StudentID  
JOIN Course  
  ON Enrollment.CourseID = Course.ID;
```

13. Subqueries

A **subquery** is a query contained inside another query.

For example, from a single table, to show students whose GPA is above the average GPA:

```
SELECT ID, Name, GPA
FROM Student
WHERE GPA > (
    SELECT AVG(GPA)
    FROM Student
);
```

Another example, show the student(s) with the highest GPA:

```
SELECT ID, Name, Major, GPA
FROM Student
WHERE GPA = (
    SELECT MAX(GPA)
    FROM Student
);
```

Another way to find the student with the highest GPA (without using a subquery) is:

```
SELECT ID, Name, Major, GPA
FROM Student
ORDER BY GPA DESC
LIMIT 1;
```

However, this returns only **one** student with the highest GPA.

We can use subquery using more than one table. For example, show all course ID and Name that have more than 3 students enrolled:

```
SELECT ID, Name  
FROM Course  
WHERE ID IN (  
    SELECT CourseID  
    FROM Enrollment  
    GROUP BY CourseID  
    HAVING COUNT(*) > 3 );
```

You might have noticed that you can use either a subquery or join when working with more than one table.

For example:

```
SELECT ID, Name
FROM Course
WHERE ID IN (
    SELECT CourseID
    FROM Enrollment
    GROUP BY CourseID
    HAVING COUNT(*) > 3 );
```

This can be written using JOIN instead:

```
SELECT Course.ID, Course.Name
FROM Course
JOIN Enrollment
    ON Course.ID = Enrollment.CourseID
GROUP BY Enrollment.CourseID
HAVING COUNT(*) > 3;
```

When working with **more than one table**, a **JOIN is generally preferable** when your goal is to combine related data from those tables.

Use a **subquery** when one query depends on the result of another, especially for comparisons or filtering:

```
SELECT Name, GPA
FROM Student
WHERE GPA > (
    SELECT AVG(GPA)
    FROM Student
);
```

A simple rule is:

JOIN → use when combining columns from multiple related tables.

Subquery → use when one query needs the result of another query.

In many cases, either approach can work, but **JOINS are usually clearer for relationships between tables.**

14. Derived Table

A **derived table** is a **temporary result set created by a subquery in the FROM clause**.

It behaves like a table that exists only while the query is running.

Example:

```
SELECT MAX(NumberOfStudents)
FROM (
    SELECT Major,
           COUNT(*) AS NumberOfStudents
    FROM Student
    GROUP BY Major
) AS MajorCounts;
```

The subquery:

```
SELECT Major,
       COUNT(*) AS NumberOfStudents
FROM Student
GROUP BY Major
```

creates the derived table. The alias:
`AS MajorCounts`

gives that temporary table a name.

A derived table requires an alias.

If you want to print the **Major name together with the maximum number of students**, you need to return the row whose count equals the maximum count.

A clean way is using CTE:

```
WITH MajorCounts AS (  
    SELECT Major,  
           COUNT(*) AS NumberOfStudents  
    FROM Student  
    GROUP BY Major  
)
```

```
SELECT Major, NumberOfStudents  
FROM MajorCounts  
WHERE NumberOfStudents = (  
    SELECT MAX(NumberOfStudents)  
    FROM MajorCounts  
);
```

15. Common Table Expressions (CTE)

A **Common Table Expression**, or **CTE**, creates a temporary named query result. A CTE begins with the keyword: WITH

Example:

```
WITH CourseCounts AS (  
    SELECT FacultyID,  
           COUNT(*) AS NumCourses  
    FROM Course  
    GROUP BY FacultyID  
)
```

```
SELECT *  
FROM CourseCounts  
WHERE NumCourses > (  
    SELECT AVG(NumCourses)  
    FROM CourseCounts  
);
```

CourseCounts exists only while the query is being executed.

Without CTE, the SQL on the left will be written as:

```
SELECT *  
FROM (  
    SELECT FacultyID,  
           COUNT(*) AS NumCourses  
    FROM Course  
    GROUP BY FacultyID  
) AS CourseCounts  
WHERE NumCourses > (  
    SELECT AVG(NumCourses)  
    FROM (  
        SELECT FacultyID,  
               COUNT(*) AS NumCourses  
        FROM Course  
        GROUP BY FacultyID  
    ) AS AvgCourseCounts  
);
```

Another example of CTE. This script shows the students who are enrolled in more than one course, along with the number of courses each student is taking.

```
WITH StudentCourseCounts AS (  
    SELECT StudentID,  
           COUNT(*) AS NumCourses  
    FROM Enrollment  
    GROUP BY StudentID  
)  
  
SELECT Student.ID,  
       Student.Name,  
       StudentCourseCounts.NumCourses  
FROM StudentCourseCounts  
JOIN Student  
    ON StudentCourseCounts.StudentID = Student.ID  
WHERE StudentCourseCounts.NumCourses > 1;
```

The same script, without using CTE would be:

```
SELECT Student.ID,  
       Student.Name,  
       StudentCourseCounts.NumCourses  
FROM (  
    SELECT StudentID,  
           COUNT(*) AS NumCourses  
    FROM Enrollment  
    GROUP BY StudentID  
) AS StudentCourseCounts  
JOIN Student  
    ON StudentCourseCounts.StudentID = Student.ID  
WHERE StudentCourseCounts.NumCourses > 1;
```

